

Reproduction Study of MK-UNet

Evaluation on ClinicDB and ColonDB

Step 1 - Deliverable 2: Technical Report

M. Mohaiminul Islam

Paper: Rahman & Marculescu, MK-UNet (arXiv:2509.18493) – Code: SLDGroup/MK-UNet at commit 2fa8b23

1 Objective and Paper Overview

This study reproduces part of the MK-UNet experiments and checks the released code against the paper. The public repository provides the polyp pipeline, so the experiments were run on ClinicDB and ColonDB using MK-UNet-T. The main goal was not only to obtain a DICE score, but also to understand the training protocol, check the model structure, and identify reproducibility issues.

MK-UNet is a five-stage U-shaped CNN for lightweight medical image segmentation [1]. The standard model is reported with 0.316 M parameters and 0.314 G FLOPs. The paper reports an average DICE of 89.75% across six datasets. The tiny model, MK-UNet-T, is reported with 0.027 M parameters and 0.062 G FLOPs. The main components are:

- MKDC: parallel depth-wise convolutions with several kernel sizes, followed by summation and channel shuffle.
- MKIR: an inverted residual block that uses MKDC for multi-kernel feature extraction.
- MKIRA: channel attention, spatial attention, and MKIR in the decoder for feature refinement.
- GAG: a grouped attention gate that filters skip-connection features before decoder fusion.

The multi-kernel design is a central part of the method. In the paper’s BUSI ablation, the DICE score increases from 70.83 with a single 1×1 kernel to 78.04 with the kernel set $\{1, 3, 5\}$ [1].

2 Reproduction Protocol

2.1 Code and environment

Training was performed on Windows 11 using an Intel i9-14900 CPU, 31.7 GB RAM, Python 3.8.20, and PyTorch 1.11.0+cu113. No GPU was used. The released code contained hard-coded CUDA calls, so six `.cuda()` calls were changed to `.to(device)`, checkpoint loading was given `map_location`, and a device option was added. Seed, run-count, dataset, data-root, augmentation parsing, and resume options were also added for controlled execution. These changes were made to support reproducible CPU runs; the architecture, loss, and metric formulas were kept unchanged.

The ClinicDB split was 489/61/62 and the ColonDB split was 303/38/38 for train/validation/test, matching the 80:10:10 split described in the paper.

2.2 Paper and repository settings

The paper and the released training script use different training settings. Table 1 shows the main differences. For the benchmark runs in this study, the repository-default training hyperparameters were kept, but only one deterministic run was used with seed 42.

Table 1: Main protocol differences between the paper and the released training script.

Setting	Paper [1]	Run used here
Learning rate	1×10^{-4}	5×10^{-4} + cosine scheduler
Batch size	16	8
Augmentation	none stated	rotation and flips enabled
Number of runs	five-run mean	one run, seed 42
Epochs	200	200
Optimizer	AdamW	AdamW

Parameter counts were checked before training. The measured values for the five variants were 0.0274 M, 0.0930 M, 0.3156 M, 1.1502 M, and 3.7581 M, which match the paper’s reported values to the shown precision. This provides a useful structural consistency check. Measured FLOPs were about 5–8% higher than the paper’s values across the variants. This consistent difference may come from the FLOP-counting method, but the exact reason was not established in this study.

3 Experimental Results

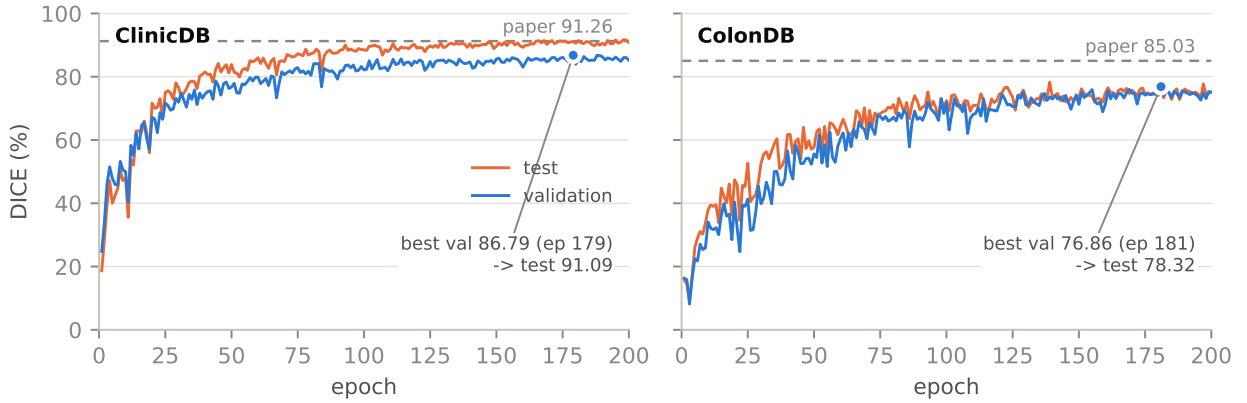


Figure 1: Training curves for 200 epochs using one run with seed 42. The checkpoint was selected using validation DICE. The released script also evaluates the test split at every epoch; test values were not used for checkpoint selection or tuning.

Both datasets used the same run configuration. ClinicDB reached a test DICE of 91.09 and IoU of 84.63. ColonDB reached a test DICE of 78.32 and IoU of 69.46. The best validation DICE values were 86.79 at epoch 179 for ClinicDB and 76.86 at epoch 181 for ColonDB.

Table 2: Published MK-UNet-T results and the single-run results obtained in this study.

Dataset	Paper DICE	Our DICE	Our IoU	Difference
ClinicDB	91.26	91.09	84.63	-0.17
ColonDB	85.03	78.32	69.46	-6.71

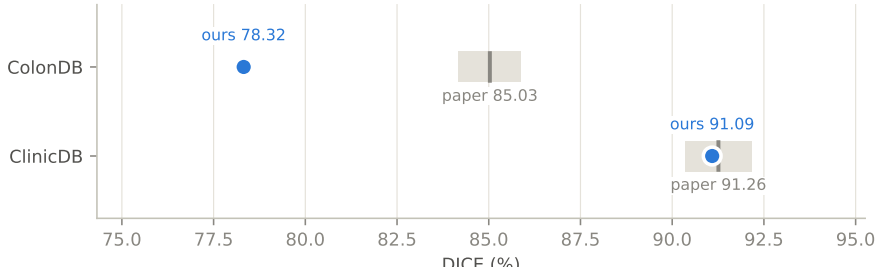


Figure 2: Published five-run means and the single-run values obtained here. The shaded range is an illustrative reference based on the upper end of the paper’s reported 1–4% run-to-run variation. It is not a confidence interval and not a dataset-specific standard deviation.

The ClinicDB result is 0.17 DICE points below the published mean. Because this study used one seed and a different training protocol from the paper, this should be described as numerically consistent with the published result, not as an exact reproduction.

The ColonDB result is 6.71 DICE points below the published mean. The paper reports 1–4% standard deviations across its five-run results, but it does not provide a separate standard deviation for ColonDB. Therefore, this study cannot establish statistical significance from the published information alone. The ColonDB training curve converges and reaches a stable plateau, which makes a major optimization failure less likely, but it does not rule out protocol, preprocessing, augmentation, or seed effects.

Three possible explanations remain untested: the smaller ColonDB training set may be more sensitive to the protocol difference; repository-default augmentation may affect the tiny model differently on ColonDB; and ColonDB may have stronger seed-to-seed variation. The most useful next diagnostic experiment is to run ColonDB with the paper’s exact settings across several seeds.

4 Codebase and Reproducibility Findings

4.1 Per-image variation

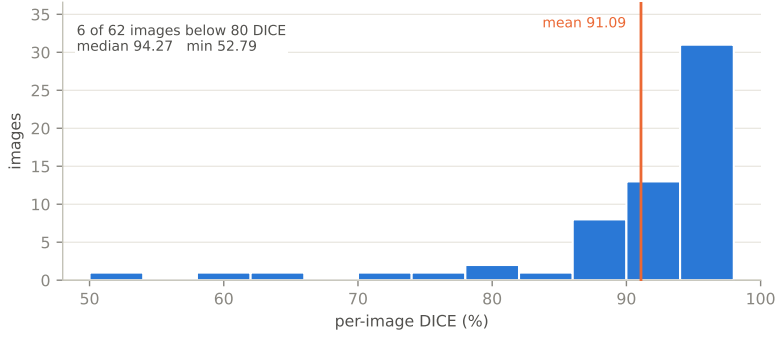


Figure 3: Per-image DICE on the ClinicDB test set. The mean is 91.09, the median is 94.27, six of 62 images are below 80, and the minimum is 52.79.

The ClinicDB mean does not show the full error pattern. Most test images score highly, but a small low-performing tail remains. This suggests that per-image analysis is useful in addition to the mean DICE score.

4.2 Unused auxiliary prediction heads

The forward pass computes four segmentation outputs, but only the final output is returned for binary segmentation. A gradient check showed that 31 parameters in `out1/out2/out3` received no gradient in this setting. These heads therefore add redundant computation during the binary runs used here. The paper recommends deep supervision for multi-class segmentation, so the same heads may be useful in a different training setting.

4.3 Narrow channel-attention bottleneck in MK-UNet-T

The released code uses channel-attention reduction ratios of 16, 16, 16, 8, and 4 across the five decoder stages. With MK-UNet-T channels [4, 8, 16, 24, 32], the hidden attention width becomes 1 in four of the five stages. This creates a very narrow attention bottleneck in the tiny model. The standard model does not become this narrow. The paper describes a single reduction ratio of 16, while the code uses stage-specific ratios. Raising the minimum hidden width to 4 would add 440 parameters, about 1.6% of MK-UNet-T’s parameter count.

The code also reuses one spatial-attention module across all decoder stages, while channel-attention modules are stage-specific. This is another implementation detail that is not clearly described in the paper.

5 Limitations and Link to the Path A Extension

This reproduction has four main limitations. Only one seed was run for each dataset, while the paper reports five-run means. Repository-default training settings were used instead of the paper-exact settings. Training was CPU-only, so runtime cannot be compared with the paper’s GPU throughput results. The checked repository version also provides only the polyp pipeline used here, so the other four benchmarks were not reproduced in this setup.

The code audit identified one direct methodological question in MKDC. The original block always computes the 1×1 , 3×3 , and 5×5 branches and uses the same sum for every input. The Step 2 Path A proposal develops this into MK-UNet-CC. It first tests input-dependent branch weights as a control, then tests predictive entropy as an extra routing signal, annealed top-1 routing that skips unused branches at inference, and routing-guided pruning for a smaller static model. Adaptive weighting itself is not treated as the main novelty. The main question is whether conditional branch execution is useful at the very small scale of MK-UNet-T and whether FLOP savings become real latency savings. The Step 2 preliminary results are reported separately and do not change the interpretation of this Step 1 reproduction.

References

- [1] M. M. Rahman and R. Marculescu, “MK-UNet: Multi-kernel Lightweight CNN for Medical Image Segmentation,” Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops, 2025, pp. 1053–1062.
- [2] SLDGroup, “MK-UNet,” public code repository, commit `2fa8b23`.